

game.c

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#include "types.h"
#include "functions.h"

int rollDice(void)
{
    int die1 = rand() % 6 + 1;
    int die2 = rand() % 6 + 1;

    return die1 + die2;
}

/* Moves a player, returns 1 if they passed or landed on GO. */
int movePlayer(GameState game[], int playerIndex, int dice)
{
    int oldPosition;
    int passedGo;
    char moneyBuffer[32];

    oldPosition = game[0].players[playerIndex].position;
    game[0].players[playerIndex].position = (oldPosition + dice) % BOARD_SIZE;

    printf("%s moves from Square %d to Square %d.\n",
           game[0].players[playerIndex].name,
           oldPosition,
           game[0].players[playerIndex].position);

    passedGo = 0;

    if(oldPosition + dice >= BOARD_SIZE)
    {
        passedGo = 1;

        receiveMoney(game, playerIndex, GO_MONEY);

        printf("%s passed GO.\n", game[0].players[playerIndex].name);

        formatLKR(GO_MONEY, moneyBuffer);

        printf("Collected LKR %s\n", moneyBuffer);

        printf("Current Balance : LKR %d\n",
               game[0].players[playerIndex].cash);
    }

    return passedGo;
}

/* Handles a player's turn while they're in jail.
Return codes:
    0 = wasn't in jail - take a normal turn
    1 = was in jail (released or not) - turn ends, no move this turn */
int handleJail(GameState game[], int playerIndex)
{
    int die1, die2;

    if(!game[0].players[playerIndex].inJail)
```

```

        return 0;

die1 = rand() % 6 + 1;
die2 = rand() % 6 + 1;

printf("%s is in Jail. Rolled %d and %d.\n",
        game[0].players[playerIndex].name, die1, die2);

if(die1 == die2)
{
    printf("Doubles! %s is released from Jail.\n",
            game[0].players[playerIndex].name);

    /* Rule 13: rolling doubles releases the player, but they do
       not move this turn - they remain on the Jail square and
       move on their next turn. */
    game[0].players[playerIndex].inJail = 0;
    game[0].players[playerIndex].jailTurns = 0;

    return 1;
}

/* Rule 13: paying bail is a strategy choice made after a failed
doubles roll. The player is released but does not move this
turn. */
if(shouldPayBail(game, playerIndex) &&
    game[0].players[playerIndex].cash >= JAIL_BAIL)
{
    printf("%s pays bail of LKR %d and is released from Jail.\n",
            game[0].players[playerIndex].name, JAIL_BAIL);

    payMoney(game, playerIndex, JAIL_BAIL);

    game[0].players[playerIndex].inJail = 0;
    game[0].players[playerIndex].jailTurns = 0;

    return 1;
}

game[0].players[playerIndex].jailTurns++;

if(game[0].players[playerIndex].jailTurns >= JAIL_MAX_TURNS)
{
    printf("%s has been in Jail for %d turns - must pay bail of "
            "LKR %d and is released.\n",
            game[0].players[playerIndex].name, JAIL_MAX_TURNS, JAIL_BAIL);

    payMoney(game, playerIndex, JAIL_BAIL);

    game[0].players[playerIndex].inJail = 0;
    game[0].players[playerIndex].jailTurns = 0;

    return 1;
}

printf("%s remains in Jail (%d/%d turns).\n",
        game[0].players[playerIndex].name,
        game[0].players[playerIndex].jailTurns,
        JAIL_MAX_TURNS);

return 1;
}

```

```

/* One player's whole turn, matching Rule 3. Returns 1 if GO was
   passed at any point during the turn. */
int playTurn(GameState game[], int playerIndex)
{
    int dice;
    int pos;
    int passedGo;
    int jailResult;
    SquareType squareType;

    if(game[0].players[playerIndex].bankrupt)
        return 0;

    printf("\n-----\n");
    printf("%s's Turn\n", game[0].players[playerIndex].name);

    /* Step 1: fix any damaged, worn, or neglected buildings first */
    tryAutoRepair(game, playerIndex);
    performMaintenance(game, playerIndex);
    renovateStructuralDamage(game, playerIndex);

    passedGo = 0;

    jailResult = handleJail(game, playerIndex);

    if(jailResult == 1)
        return 0;    /* was in jail - released or not, no move this turn */

    /* jailResult == 0: wasn't in jail - take a completely normal turn. */
    dice = rollDice();
    printf("%s rolled %d.\n", game[0].players[playerIndex].name, dice);

    if(movePlayer(game, playerIndex, dice))
        passedGo = 1;

    /* Step 4: resolve landing action */
    pos = game[0].players[playerIndex].position;
    squareType = game[0].board[pos].type;

    if(squareType == PROPERTY || squareType == RAILWAY || squareType == UTILITY)
    {
        payRent(game, playerIndex, dice);
        buyProperty(game, playerIndex);
    }
    else if(squareType == EVENT)
    {
        executeEvent(game, playerIndex);
    }
    else if(squareType == TAX)
    {
        payTax(game, playerIndex);
    }
    else if(squareType == COMMUNITY_FUND)
    {
        payCommunityFundTax(game, playerIndex);
    }
    else if(squareType == GO_TO_JAIL)
    {
        printf("%s is sent to Jail!\n", game[0].players[playerIndex].name);
        game[0].players[playerIndex].position = JAIL_SQUARE;
        game[0].players[playerIndex].inJail = 1;
        game[0].players[playerIndex].jailTurns = 0;
    }
}

```

```

else if(squareType == BANK)
{
    handleBankVisit(game, playerIndex);
}
else if(squareType == INSURANCE)
{
    handleInsuranceVisit(game, playerIndex);
}
/* GO, JAIL (just visiting), FREE_PARKING need no action */

/* Step 6: build if eligible */
constructBuildings(game, playerIndex);

/* Section 3.4: the Opportunistic Trader first dumps any property
that economic events have marked for a decline. */
sellDecliningProperties(game, playerIndex);

/* Step 7: mortgage or redeem if the player's situation calls for it */
handleMortgageDecisions(game, playerIndex);

return passedGo;
}

/* Resolves one group of (possibly tied) players into a strict order.
Writes the result into output[] starting at startIndex, and
returns how many players it wrote. Any players still tied after
rolling call this function again on just themselves. */
int resolveGroup(GameState game[], int groupPlayers[], int groupSize,
                int output[], int startIndex)
{
    int rolls[MAX_PLAYERS];
    int i, j, temp;
    int runStart, runLength;
    int tiedPlayers[MAX_PLAYERS];
    int written;
    int k;

    written = 0;

    for(i = 0; i < groupSize; i++)
    {
        rolls[i] = rollDice();
        printf("%s rolls %d.\n", game[0].players[groupPlayers[i]].name, rolls[i]);
    }

    /* sort this group by roll, highest first */
    for(i = 0; i < groupSize - 1; i++)
    {
        for(j = 0; j < groupSize - 1 - i; j++)
        {
            if(rolls[j] < rolls[j + 1])
            {
                temp = rolls[j];
                rolls[j] = rolls[j + 1];
                rolls[j + 1] = temp;

                temp = groupPlayers[j];
                groupPlayers[j] = groupPlayers[j + 1];
                groupPlayers[j + 1] = temp;
            }
        }
    }
}

```

```

/* walk the sorted list - a run of equal rolls is a tie */
i = 0;

while(i < groupSize)
{
    runStart = i;
    runLength = 1;

    while(runStart + runLength < groupSize &&
        rolls[runStart + runLength] == rolls[runStart])
    {
        runLength++;
    }

    if(runLength == 1)
    {
        output[startIndex + written] = groupPlayers[runStart];
        written++;
    }
    else
    {
        printf("Tie detected between %d players at %d. Rolling again.\n",
            runLength, rolls[runStart]);

        for(k = 0; k < runLength; k++)
            tiedPlayers[k] = groupPlayers[runStart + k];

        written += resolveGroup(game, tiedPlayers, runLength,
            output, startIndex + written);
    }

    i = runStart + runLength;
}

return written;
}

void determineTurnOrder(GameState game[], int turnOrder[])
{
    int allPlayers[MAX_PLAYERS];
    int i;

    for(i = 0; i < MAX_PLAYERS; i++)
        allPlayers[i] = i;

    printf("\nDetermining the First Player\n");

    resolveGroup(game, allPlayers, MAX_PLAYERS, turnOrder, 0);

    printf("\n%s will begin the game.\n", game[0].players[turnOrder[0]].name);

    printf("\nTurn order:\n");
    for(i = 0; i < MAX_PLAYERS; i++)
        printf("%s\n", game[0].players[turnOrder[i]].name);
}

int countHouses(GameState game[], int playerIndex)
{
    int i;
    int count;

    count = 0;

```

```

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == PROPERTY &&
            game[0].board[i].property.owner == playerIndex &&
            !game[0].board[i].property.hotel)
        {
            count += game[0].board[i].property.houses;
        }
    }

    return count;
}

int countHotels(GameState game[], int playerIndex)
{
    int i;
    int count;

    count = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == PROPERTY &&
            game[0].board[i].property.owner == playerIndex &&
            game[0].board[i].property.hotel)
        {
            count++;
        }
    }

    return count;
}

void displayRoundSummary(GameState game[], int round)
{
    int i;

    printf("\n=====\\n");
    printf("Round %d Summary\\n", round);
    printf("=====\\n");

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        char cashBuf[32];
        char netBuf[32];
        char loanBuf[32];

        formatLKR(game[0].players[i].cash, cashBuf);
        formatLKR(calculateNetWorth(game, i), netBuf);

        printf("%s\\n", game[0].players[i].name);
        printf("Cash : LKR %s\\n", cashBuf);
        printf("Net Worth : LKR %s\\n", netBuf);
        printf("Properties : %d\\n", game[0].players[i].propertiesOwned);
        printf("Railways : %d\\n", game[0].players[i].railwaysOwned);
        printf("Utilities : %d\\n", game[0].players[i].utilitiesOwned);
        printf("Houses : %d\\n", countHouses(game, i));
        printf("Hotels : %d\\n", countHotels(game, i));

        if(game[0].players[i].loan.active)
        {
            formatLKR(game[0].players[i].loan.amount, loanBuf);
            printf("Outstanding Loan : LKR %s\\n", loanBuf);
        }
    }
}

```

```

    }
    else
    {
        printf("Outstanding Loan : None\n");
    }

    if(game[0].players[i].bankrupt)
        printf("Status : BANKRUPT\n");

    printf("-----\n");
}
}

```

```

int countSolventPlayers(GameState game[])
{
    int i;
    int count;

    count = 0;

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        if(!game[0].players[i].bankrupt)
            count++;
    }

    return count;
}

```

```

/* A round is not "each player took one turn" - it only finishes
   once every player still in the game has passed GO at least once.
   Since players move different distances each turn, this usually
   takes several turns per player, not one - so this loop keeps
   cycling through turnOrder until everyone has crossed GO. */
void playGame(GameState game[], int turnOrder[])
{

```

```

    int round;
    int turnPointer;
    int playerIndex;
    int passedGoThisRound[MAX_PLAYERS];
    int i;
    int allPassed;
    int passedGo;

    round = 1;
    turnPointer = 0;

    for(i = 0; i < MAX_PLAYERS; i++)
        passedGoThisRound[i] = 0;

    printf("\n=====\\n");
    printf("ROUND %d\\n", round);
    printf("=====\\n");

    while(round <= MAX_ROUNDS)
    {
        playerIndex = turnOrder[turnPointer];

        if(!game[0].players[playerIndex].bankrupt)
        {
            passedGo = playTurn(game, playerIndex);

            if(passedGo)

```

```

        passedGoThisRound[playerIndex] = 1;
    }

    turnPointer = (turnPointer + 1) % MAX_PLAYERS;

    /* has every player still playing passed GO this round? */
    allPassed = 1;

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        if(game[0].players[i].bankrupt)
            continue;

        if(!passedGoThisRound[i])
        {
            allPassed = 0;
            break;
        }
    }

    if(!allPassed)
        continue;

    processLoans(game);
    processInsuranceExpiry(game);
    ageProperties(game);
    ageBuildings(game);
    decrementEventTimers(game);
    decrementModifiers(game);
    enforceAntiSpeculation(game);

    if(round % ECONOMY_CYCLE == 0)
    {
        triggerDisaster(game);
        applyInflation(game);
        reviewPropertyMarket(game, round);
    }

    if(round % EVENT_CYCLE == 0)
    {
        triggerEconomicEvent(game);
        drawRegionalCard(game);
    }

    if(round % REGULATION_CYCLE == 0)
    {
        triggerGovernmentRegulation(game);
    }

    displayRoundSummary(game, round);
    displayMarketConditions(game);

    if(countSolventPlayers(game) <= 1)
    {
        printf("\nOnly one solvent player remains. Ending game.\n");
        break;
    }

    round++;

    if(round <= MAX_ROUNDS)
    {
        printf("\n=====");
    }

```

```

        printf("ROUND %d\n", round);
        printf("=====\n");
    }

    for(i = 0; i < MAX_PLAYERS; i++)
        passedGoThisRound[i] = 0;
}

/* Whoever is still solvent wins. If several are still solvent at
   round 500, the highest net worth wins (Rule 15). */
int determineWinner(GameState game[])
{
    int i;
    int winner;
    int bestNetWorth;
    int netWorth;

    winner = -1;
    bestNetWorth = 0;

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        if(game[0].players[i].bankrupt)
            continue;

        netWorth = calculateNetWorth(game, i);

        if(winner == -1 || netWorth > bestNetWorth)
        {
            winner = i;
            bestNetWorth = netWorth;
        }
    }

    if(winner == -1)
    {
        /* edge case: everyone bankrupt at once - compare everyone anyway */
        for(i = 0; i < MAX_PLAYERS; i++)
        {
            netWorth = calculateNetWorth(game, i);

            if(winner == -1 || netWorth > bestNetWorth)
            {
                winner = i;
                bestNetWorth = netWorth;
            }
        }
    }

    return winner;
}

void displayFinalResults(GameState game[])
{
    int winner;
    int i;
    char cashBuf[32];
    char propBuf[32];
    char loanBuf[32];
    char netBuf[32];

    winner = determineWinner(game);

```

```

printf("\n=====\\n");
printf("GAME OVER\\n");
printf("=====\\n");

printf("Winner\\n%s\\n", game[0].players[winner].name);

formatLKR(game[0].players[winner].cash, cashBuf);
printf("Total Cash\\nLKR %s\\n", cashBuf);

formatLKR(calculatePropertyValue(game, winner) +
          calculateBuildingValue(game, winner), propBuf);
printf("Total Property Value\\nLKR %s\\n", propBuf);

if(game[0].players[winner].loan.active)
{
    formatLKR(game[0].players[winner].loan.amount, loanBuf);
    printf("Outstanding Loans\\nLKR %s\\n", loanBuf);
}
else
{
    printf("Outstanding Loans\\nNone\\n");
}

formatLKR(calculateNetWorth(game, winner), netBuf);
printf("Net Worth\\nLKR %s\\n", netBuf);
printf("=====\\n");

printf("\\nFinal Standings (all players)\\n");
printf("=====\\n");

for(i = 0; i < MAX_PLAYERS; i++)
{
    printf("\\n%s\\n", game[0].players[i].name);
    printf("Cash : LKR %d\\n", game[0].players[i].cash);
    printf("Net Worth : LKR %d\\n", calculateNetWorth(game, i));
    printf("Properties : %d\\n", game[0].players[i].propertiesOwned);
    printf("Railways : %d\\n", game[0].players[i].railwaysOwned);
    printf("Utilities : %d\\n", game[0].players[i].utilitiesOwned);
    if(game[0].players[i].bankrupt)
    {
        printf("Status : Bankrupt\\n");
    }
    else
    {
        printf("Status : Active\\n");
    }
}
}

void startGame(GameState game[])
{
    int turnOrder[MAX_PLAYERS];
    char moneyBuffer[32];

    srand((unsigned)time(NULL));

    initializeBoard(game);
    initializePlayers(game);
    initEconomy(game);
    initMarket(game);

    printf("Player 1 : %s\\n", game[0].players[0].name);

```

```
printf("Player 2 : %s\n", game[0].players[1].name);
printf("Player 3 : %s\n", game[0].players[2].name);
printf("Player 4 : %s\n", game[0].players[3].name);

formatLKR(START_MONEY, moneyBuffer);
printf("\nEach player begins with LKR %s.\n", moneyBuffer);

determineTurnOrder(game, turnOrder);

playGame(game, turnOrder);

displayFinalResults(game);
}
```